Komputerowo Zintegrowane Wytwarzanie
Sprawozdanie nr 2 : WiTi

Mikołaj Nowak 280082

17 kwietnia 2026

---

1 Zasada działania Algorytmu Programowania Dynamicznego

1. Rozpoczynamy rekurencyjny algorytm od pustych uszeregowań.
2. Algorytm wykonuje następujące kroki dla każdego badanego uszeregowania:
   (a) Sprawdzamy czy optymalny wynik został już policzony, jeśli tak zwracamy go i przerywamy dalsze obliczenia.
   (b) Iterujemy poprzez wszystkie dostępne operacje, czyli takie które nie zostały oznaczone jako wykonane w masc e.
   (c) Wyznaczamy dla nowej operacji czas oraz karę za spóźnienie.
   (d) Sprawdzamy optymalny koszt ułożenia reszty zadań poprzez rekurencyjne wywołanie algorytmu.
   (e) Wybieramy wariant o najmniejszym koszcie i zapamiętujemy najlepsze uszeregowanie z wynikiem.
3. Analogicznie postępujemy aż wszystkie możliwośc zostaną policzone i zapisane w tabeli.

2 Kod Algorytmu Programowania Dynamicznego

```cpp

pair<int, vector<operacja>> algorytm(vector<operacja> &operacje, int ocena_0, int maska)
{
    int ilo_operacji = operacje.size();
    if (maska == (1 << ilo_operacji) - 1)
    {
        return {0, {}};
    }
    if (mapa_optymalnych_czasow.count(maska))
    {
        return mapa_optymalnych_czasow[maska];
    }
    int najmniejsza_ocena = 99999999;
    vector<operacja> najlepsza_kolejnosc;
    for (int i = 0; i < ilo_operacji; i++)
    {
        int obecna_operacja = (1 << i);
        if ((maska & obecna_operacja) == 0)
        {
            int C = ocena_0 + operacje[i].p;
            int K = operacje[i].w * max(0, C - operacje[i].d);

            auto wynik = algorytm(operacje, C, maska | (1 << i));
            if (wynik.first + K < najmniejsza_ocena)
            {
                najmniejsza_ocena = wynik.first + K;
                najlepsza_kolejnosc = wynik.second;
                najlepsza_kolejnosc.insert(najlepsza_kolejnosc.begin(), operacje[i]);
            }
        }
    }
    mapa_optymalnych_czasow[maska] = {najmniejsza_ocena, najlepsza_kolejnosc};
    return mapa_optymalnych_czasow[maska];
}
}
```

Listing 1: Kod C++ Algorytmu Programowania Dynamicznego

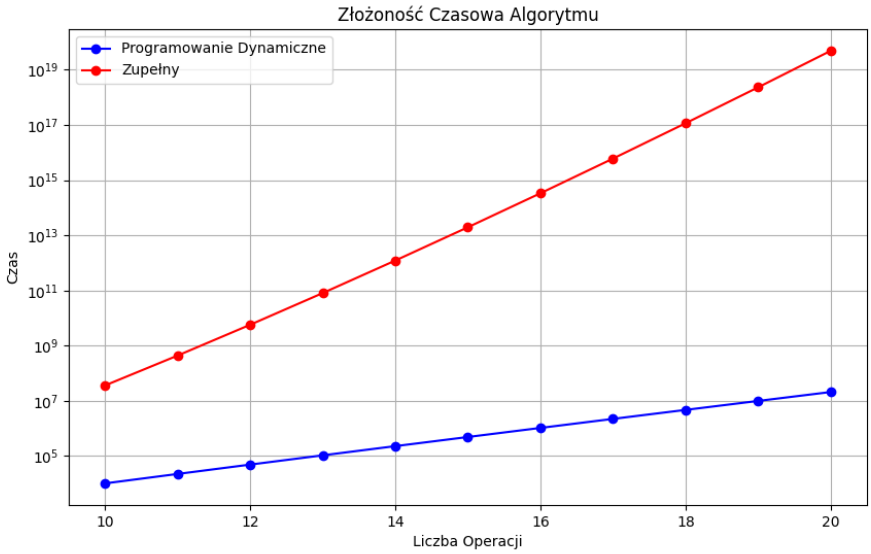
3 Kod Algorytmu Przeszukania Zupełnego

```cpp
    std::pair<int, vector<operacja>> algorytm_przegl d_zupe ny(vector<operacja> operacje, int ocena_0, vector<operacja> kolejnosc)

{
    int ilosc_operacji = operacje.size();
    if (ilosc_operacji == 0)
    {
        return {0, kolejnosc};
    }
    int najmniejsza_ocena = 99999999;
    vector<operacja> najlepsza_kolejnosc = kolejnosc;
    for (int i = 0; i < ilosc_operacji; i++)
    {
        int C = ocena_0 + operacje[i].p;
        int K = operacje[i].w * max(0, C - operacje[i].d);

        vector<operacja> operacje_kopia = operacje;
        operacje_kopia.erase(operacje_kopia.begin() + i);

        vector<operacja> kolejnosc_kopia = kolejnosc;
        kolejnosc_kopia.push_back(operacje[i]);

        auto [ocena, kolejnosc_wykonania] = algorytm_przegl d_zupe ny(operacje_kopia, C, kolejnosc_kopia);
        najmniejsza_ocena = min(najmniejsza_ocena, ocena + K);
        if (najmniejsza_ocena == ocena + K)
        {

            najlepsza_kolejnosc = kolejnosc_wykonania;
        }
    }

    return {najmniejsza_ocena, najlepsza_kolejnosc};
}
```

Listing 2: Kod C++ Algorytmu Przeszukania Zupełnego

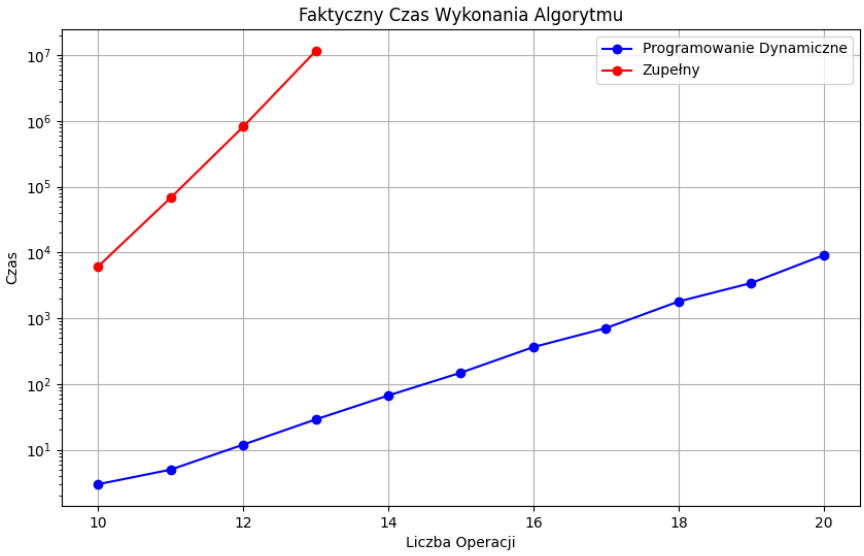
4 Czas Wykonania

4.1 Czas Teoretyczny

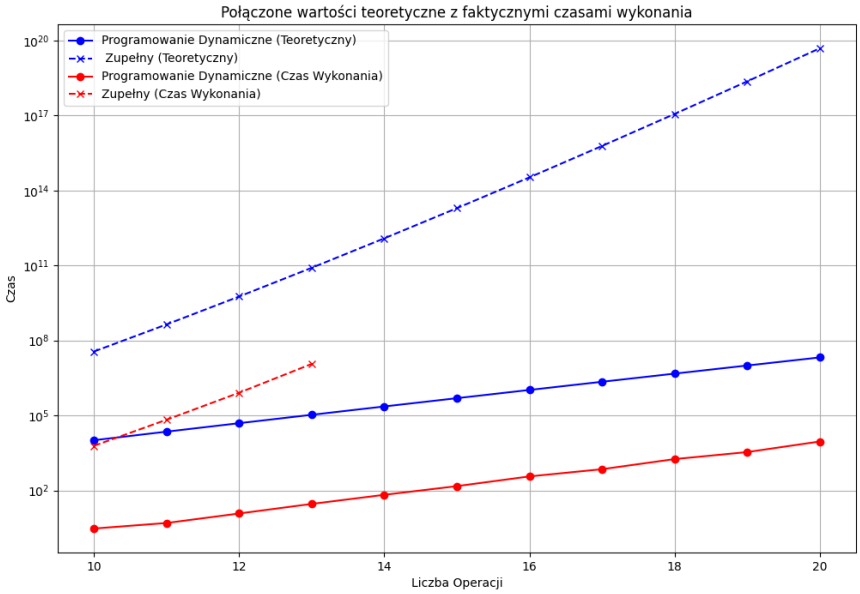


Rysunek 1: Czasy teoretyczne: $n * 2^n$ (niebieski) programowanie dynamiczne oraz $n * n!$ (czerwony) dla przeszukania zupełnego

4.2 Czas Wykonywania Kodu



Rysunek 2: Czasy wykonania dla obu algorytmów.



Rysunek 3: Porównanie teoretycznych złożoności do faktycznych wyników.

Liczba operacji (n)	**Czas wykonania (PD)**	**Czas wykonania (Przegląd zupełny)**
10	3	6111
11	5	67648
12	12	813839
13	29	11561190
14	67	–
15	149	–
16	365	–
17	709	–
18	1799	–
19	3430	–
20	9089	–

Tabela 1: Porównanie rzeczywistego czasu wykonania algorytmów w zależności od liczby operacji.